

Curated File Guide: script.js

The public website runtime that renders the portfolio, search, project windows, browser history, rich content, links, and AI chat behavior.

Before The Code: File Overview

Abstract

script.js is the public website runtime. It is the JavaScript that turns the static portfolio files into an interactive recruiter-facing site. The HTML page provides stable containers and the JSON catalog provides saved content, but script.js decides what appears, how projects are filtered, how rich text renders, how files become links, how project sections open into full-window views, how browser Back and Refresh interact with those views, how search behaves like a real search tool, and how the AI chat gathers useful context before asking the backend.

The important distinction is that script.js is public and read-only. It should never expose builder-only editing controls. Its job is to present what has already been saved and parsed by the builder. That is why much of the file is about rendering, normalization, routing, search indexing, assistant context selection, and UI state rather than file writes.

Introduction

When a visitor opens the portfolio, index.html loads styles.css, electronics-search.js, and script.js. script.js immediately grabs the DOM elements it will control: the project grid, search input, filters, profile fields, resume area, contact area, AI assistant, and footer. Then it waits for portfolio data. The data can come from an embedded window.__PORTFOLIO_CATALOG__ object in previews or from projects.json on the published website. Once the catalog is loaded, script.js normalizes profile data, renders the hero and contact information, creates category filters, renders project cards, builds the search index, and restores any deep-linked project section.

This file is the main reason the same portfolio can feel alive without requiring a database-backed website. It uses static files as the source of truth, then builds an application-like experience in the browser. That makes the public website easy to host on GitHub Pages or Cloudflare, while still giving visitors search, project windows, AI questions, and responsive navigation.

Background Information

The file works with a catalog generated by the builder parser. That catalog contains categories, projects, profile fields, site content, field styles, rich text blocks, fun facts, and optional site sections. The functions in script.js assume the catalog is already safe enough to display, but they still escape HTML where user-facing strings are inserted into the DOM. Rich text rendering is intentionally structured because portfolio content can include plain text, links, formulas, images, files, code blocks, captions, and nested project subsections.

The website also has a windowing model. Project cards appear inside categories. A project section button opens a full-screen dialog. Inside that dialog, subsections can be clicked like nested folders. Browser history is updated with the project id, section index, and resource path so the browser Back button can move through the same hierarchy. This makes project content feel more like a navigable portfolio than a long flat webpage.

Search is also broader than a normal text filter. The search index contains page sections, project names, category labels, tools, languages, rich text, files, URLs, captions, uploaded text files where fetchable, and electronics keyword expansions. The goal is that a recruiter can type an electronics phrase and land near the relevant artifact, even if the phrase appears inside a project section or file caption rather than on the top card.

Because the website is static, script.js must do work that a database-backed application might normally ask a server to do. It must decide which categories exist, which projects belong to each category, which sections are empty, which files should render as downloads, which links should open externally, and which project paths should be represented in browser history. The price of static hosting is that the browser runtime has to be disciplined. The benefit is that the public site remains easy to deploy, cache, mirror, and serve from GitHub Pages or Cloudflare without a long-running custom web server.

The file also sits between several other files. index.html gives it DOM anchors. styles.css gives visual meaning to the classes it writes. electronics-search.js can provide expanded electronics vocabulary. projects.json provides the portfolio catalog. The AI backend endpoint gives it remote intelligence when available. If one of those contracts changes, script.js is usually the first file that reveals the mismatch because rendering, search, or chat stops behaving correctly.

Purpose Of The File

The purpose of script.js is to convert portfolio data into a polished public experience. The file decides not only what to render, but also what not to render. Empty sections are hidden. Overview content appears as the first readable content when present. Links that look like websites open as websites rather than downloads. Local files become downloadable resources. Images render inline only when they are meant to be shown. AI links are only displayed when related to the question context.

Without script.js, index.html would be mostly a static shell with placeholders. There would be no dynamic project directory, no generated categories, no rich project windows, no browser-aware section routing, no searchable uploaded file text, no AI context gathering, no responsive project rendering, and no dynamic profile/contact population.

Primary Responsibilities

The first responsibility is state hydration. loadProjectCatalog reads the portfolio catalog, normalizes the data, stores it in module-level variables, and triggers rendering. This is the startup path for the public website.

The second responsibility is rendering. Functions such as renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, projectCard, parsedSection, and parsedNodeContent turn structured catalog objects into actual HTML.

The third responsibility is interaction. The dialog functions create the full-window project section view, the routing functions keep browser history synchronized, the search functions build and display selectable results, and the AI functions manage the chat panel.

The fourth responsibility is interpretation. The assistant functions classify questions, gather relevant portfolio context, select public source links, decide whether a question is general or portfolio-specific, and prepare the request sent to the AI backend. This prevents the assistant from blindly dumping generic links for every question.

The fifth responsibility is restraint. Public visitors should see only finished, parsed, saved portfolio output. The file therefore hides empty content, avoids exposing builder edit controls, limits which sources are shown with AI answers, and uses normalized link behavior so a website link does not become a fake downloadable file. This restraint is what separates the public website from the local builder application.

The sixth responsibility is mobile and desktop continuity. The same project data must read cleanly on an iPhone and on a wide monitor. The responsive helper functions estimate project density and media presence, while the dialog and assistant functions keep windows and chat panels usable without letting one part of the page resize unrelated parts. script.js is therefore not only a renderer; it is also the public interaction policy for different devices.

Important Data Objects And Why They Matter

categories, projects, siteSections, profile, siteContent, funFacts, fieldStyles, and the rich variants of those fields are the hydrated catalog state. They are not hardcoded content. They are populated from projects.json or preview data, which is why the same script can run for Maurice's site and also for someone else's freshly installed builder output.

searchableEntries is the in-memory search index. Every item in it has a title, type, context, text, optional project id, optional section index, optional URL, and normalized text. The search functions score and display these entries.

assistantChatHistory and assistantSourceMap are the AI interface memory. assistantChatHistory keeps recent conversation so follow-up questions make sense. assistantSourceMap maps clickable source buttons in an answer back to portfolio search results so a visitor can jump to the relevant project or file.

currentSearchResults and searchResultLimit are the active search session. They remember what the latest query produced and how many items the user wants to see. Without these values, pressing Enter, selecting a result, or changing the result limit would not know which result list is current.

sectionRouteKey and sectionRouteParams are the browser-history contract for project windows. They keep the public URL connected to the currently opened project section and nested subsection. The values are small, but they are what let Refresh reopen the same project view and Back return to the previous view.

Code Walkthrough

The walkthrough below groups related functions where they form one idea. Small helpers are explained together when that is clearer than pretending each one is a separate chapter. Larger functions receive direct explanations of their parameters, internal objects, side effects, return values, and why they are necessary.

Page state, profile, hero, and fun facts

This group turns catalog-level identity and front-page content into visible website content.

normalize(value), normalizeFunFacts(value), and renderFunFacts()

normalize gives the rest of the file a consistent lowercase comparison string. normalizeFunFacts accepts either an array or newline-separated text, trims it, removes empty lines, and caps the visible list. renderFunFacts decides whether the fun facts callout is hidden, then renders either rich blocks or plain short facts. Without this group, top-page personal statements would be inconsistent and could show empty containers.

normalizeSiteContent, normalizeProfile, renderSiteContent, and renderProfile

These functions are the identity hydrators. normalizeSiteContent and normalizeProfile merge catalog data with safe defaults. renderSiteContent updates the hero eyebrow, title, copy, and brand text. renderProfile updates the contact band, links, email/phone behavior, resume section, hero image, brand image, profile photo, and footer owner. These functions make the same HTML shell become a personalized portfolio.

mailComposeLink(email) and phoneLink(phone)

These two helpers convert contact values into browser actions. mailComposeLink creates a Gmail compose URL with the recipient filled in. phoneLink creates a tel: URL so phones and desktop calling apps can offer a call. They are separate functions because email and phone values must be encoded differently.

Dialog dragging, resizing, and window feel

These functions manage draggable section dialogs on the public site. They turn a modal into a controlled full-window or movable document-style view.

clampSectionDialogPosition, anchorSectionDialog, and anchorSectionDialogForResize

clampSectionDialogPosition keeps a dialog within the viewport. anchorSectionDialog converts the browser's current layout position into explicit left/top coordinates so dragging can begin smoothly. anchorSectionDialogForResize does the same before resizing. Their inputs are coordinates and the dialog element. Their side effect is CSS positioning.

beginSectionDialogDrag, moveSectionDialogDrag, endSectionDialogDrag, beginSectionDialogResize, moveSectionDialogResize, and endSectionDialogResize

These functions form pointer-state machines. The begin functions remember the starting mouse/touch point and dialog rectangle. The move functions calculate deltas and update style. The end functions clear active state. They are separate because browser pointer events arrive over time, so the file must remember what operation is active between events.

updateSectionDialogMinimize, toggleSectionDialogMinimized, and enableSectionDialogDrag

These functions keep the dialog's minimized state and interaction affordances in sync. The minimized state changes classes and visible behavior; enableSectionDialogDrag wires the required event handlers once. Without them, the section window would feel like a static modal instead of a controllable portfolio window.

Rich content, links, math, code, and images

This group renders the mixed content created by the builder: text, formulas, links, images, captions, file downloads, and syntax-highlighted code.

escapeHtml, renderInlineMath, renderMultilineInlineText, sanitizeRichInlineHtml, and linkifyRichTextNodes

These functions protect and enrich displayed text. `escapeHtml` prevents raw text from becoming HTML. `renderInlineMath` recognizes formula-like fragments and renders them cleanly. `renderMultilineInlineText` preserves intentional line breaks. `sanitizeRichInlineHtml` removes unsafe tags while keeping allowed inline formatting. `linkifyRichTextNodes` turns recognized URLs into links. Together they keep rich text useful without trusting arbitrary pasted HTML.

normalizeLinkTarget, looksLikeBareWebAddress, looksLikeWebOrContactText, isWebsiteLinkItem, linkAttributes, downloadAttribute, and resourceLink

These functions decide whether a string is a web link, contact link, local download, or empty planned asset. `normalizeLinkTarget` is the main converter; `linkAttributes` adds target and rel for real web links; `downloadAttribute` adds download only for local file assets; `resourceLink` builds the final anchor or muted placeholder. This is the group that prevents LinkedIn or GitHub links from being treated like files.

normalizeCodeLanguage, detectCodeLanguage, tokenizedCodeHtml, and renderRichCodeBlock

These functions make saved code look like code instead of ordinary paragraph text. `detectCodeLanguage` uses aliases and source clues. `tokenizedCodeHtml` highlights common language tokens. `renderRichCodeBlock` wraps the result in a presentable block with language metadata. The functions do not compile code; they render code evidence for public viewing.

richImageCropStyle, richImageDownloadLink, cleanRichImageTitle, renderRichContent, and renderRichFieldContent

These functions render rich editor blocks from the builder. Images may have crop settings, captions, downloadable behavior, or visible inline behavior. `renderRichContent` loops through saved blocks and dispatches to text, image, formula, code, or file rendering. `renderRichFieldContent` applies the same idea to profile/front-page fields. This is where builder-created rich content becomes portfolio-visible content.

Project templates, categories, and responsive project cards

This group maps saved project appearance and project data into project cards that adapt to the visitor's device.

canonicalTemplateId, projectTemplateId, projectTemplateClass, projectTemplateStyle, and applyProjectTemplateToElement

These functions treat templates as appearance choices, not content templates. `canonicalTemplateId` handles legacy template names. `projectTemplateId` extracts the active appearance. `projectTemplateClass` and `projectTemplateStyle` turn that choice into CSS classes and variables. `applyProjectTemplateToElement` applies those variables to project cards or section windows. This keeps project visuals consistent between card view and opened section view.

hydrateProjectCategories, normalizeCategory, setActiveFilter, bindFilterButtons, renderCategoryFilters, categorySection, and projectCard

These functions build the project directory. `hydrateProjectCategories` combines saved categories with projects so empty and custom categories behave correctly. `renderCategoryFilters` creates filter buttons. `categorySection` renders each category band. `projectCard` renders either the modern parsed project model or the older legacy project model. The result is a page organized by categories without hardcoding category names.

inferResponsiveProfile and projectResponsiveProfile

These functions read the project structure and estimate whether the project is simple, balanced, or dense. They count visible sections, nested depth, media presence, and child nodes. The returned profile gives CSS guidance such as content width, card layout, media max width, and touch targets. This is why the website can render projects differently on phones and PCs without Codex manually rearranging each project.

Search engine behavior

This group turns portfolio content and project evidence into a local searchable index with scoring, highlighting, and result navigation.

flattenSearchText, richTextTerms, addSearchEntry, searchSnippet, entryScore, and searchResultsFor

These functions define a search entry and how it ranks. `flattenSearchText` collapses arrays, objects, and strings into searchable text. `richTextTerms` extracts text-like values from rich blocks. `addSearchEntry` stores normalized text. `entryScore` rewards exact title matches, starts-with matches, context hits, type hits, word hits, and useful kinds such as files or sections. `searchResultsFor` scores, sorts, and de-duplicates entries. The user experiences this as a real search dropdown rather than a simple project-name filter.

filesBrowserSearchable and indexSearchableFileText

`filesBrowserSearchable` decides which uploaded or linked files can be fetched as text in the browser. `indexSearchableFileText` fetches those files, appends a bounded amount of file text to the entry, and refreshes the dropdown. This allows code, logs, CSV, Markdown, Verilog, and similar files to become searchable by content when the browser can access them.

addProjectSearchEntries, addSiteSectionSearchEntries, rebuildSearchIndex, renderSearchResults, updateSearchDropdown, and goToSearchResult

These functions build and use the full index. Projects contribute titles, categories, tools, languages, sections, files, rich text, and electronics vocabulary. Site sections contribute profile/resume/contact/process content. `renderSearchResults` shows the selectable list; `updateSearchDropdown` highlights visible text; `goToSearchResult` scrolls, opens a section window, or opens a link depending on result kind.

Portfolio AI context and conversation

This group decides what the public AI assistant should know for a question and how the chat should behave.

assistantQuestionIntent and its classifier helpers

`assistantQuestionIntent` uses helpers such as `assistantQuestionIsCasual`, `assistantQuestionHasPortfolioIntent`, `assistantQuestionLooksConceptual`, and `assistantQuestionIsEngineeringRelated`. The output can be `general_conversation`, `general_engineering`, `general_knowledge`, or `portfolio_specific`. That intent decides whether the assistant should answer generally or gather portfolio context first. This prevents a question like 'what is embedded systems' from immediately dumping Maurice's project links.

assistantContextForQuestion, assistantProjectEvidence, assistantPublicProfileLinks, assistantSourcesForDisplay

These functions assemble the request context. They identify named projects, collect matching search results, gather public profile links, include project evidence, and choose which source buttons should display. `assistantSourcesForDisplay` is deliberately strict so the assistant does not append random generic links.

assistantGeneralEngineeringAnswer and assistantLocalAnswer

These provide local fallback intelligence. `assistantGeneralEngineeringAnswer` contains direct explanations for embedded systems, op amps, VCO/PWM, FPGA/ASIC, STM32, PCB, and power topics. `assistantLocalAnswer` handles greetings, identity questions, catalog-loading cases, no-match cases, and summary answers from local search results. This keeps the assistant conversational even when the backend AI is unavailable.

askRemoteAssistant and answerAssistantQuestion

`askRemoteAssistant` sends question, context, conversation, intent, and web-search permission to the backend endpoint with a timeout. `answerAssistantQuestion` is the UI orchestrator: it appends the user message, remembers conversation, shows a pending assistant message, decides intent, gathers sources, calls the remote

assistant, falls back locally if needed, replaces the pending answer, remembers the assistant response, and restores input focus. It is async because network AI calls and source fetches take time.

appendAssistantMessage, replaceAssistantMessage, renderAssistantAnswerContent, clearAssistantChat, and updateAssistantPanelGrowth

These functions are the chat interface. renderAssistantAnswerContent converts ChatGPT-like plain text into paragraphs, bullets, and code blocks. append/replace message functions update the DOM. clearAssistantChat resets history and source maps. updateAssistantPanelGrowth grows the AI console upward on desktop without resizing the contact panel. This group makes the assistant feel like a clean chatbot instead of a static Q&A box.

Parsed project windows and browser routing

This group turns nested project sections into clickable windows and makes browser Back/Forward participate.

nodeHasRenderableContent, sectionHasRenderableContent, nodeOverviewDetails, parsedNodeContent, parsedSectionContent, parsedSection, and projectCard

These functions decide what exists on the public site. Empty nodes are not rendered. Overview content is shown automatically and collapsibly when it exists. Child sections become clickable cards. Files become inline file links. projectCard wraps all of that into the category card. This group enforces the rule that only valid created content appears.

pathToString, pathFromString, nodeAtPath, nodeChildren, sectionRouteState, sectionRouteUrl, sectionRouteFromLocation, and sameSectionRoute

These are routing helpers for nested project windows. A path such as 0.2.1 means 'walk through child indexes to reach a subsection.' The route functions translate project id, section index, and resource path into URL/search/history state and back. They are necessary because the browser Back button must know which project window is currently open.

ensureSectionDialog, openParsedSection, closeSectionDialog, closeOrStepBackSectionDialog, applySectionRoute, and restoreSectionRouteAfterCatalogLoad

ensureSectionDialog creates the reusable dialog. openParsedSection finds the project/section/node, applies the project appearance template, updates route state, writes the title, renders content, and opens the full-window dialog. close/step-back functions decide whether to go up one nested level or close to the main page. applySectionRoute and restoreSectionRouteAfterCatalogLoad connect this behavior to browser history and page refresh.

loadProjectCatalog

loadProjectCatalog is the startup pipeline. It loads either embedded preview data or projects.json, stores catalog pieces in state, renders profile/site/facts/sections/filters/projects, rebuilds search, updates the search dropdown, and restores deep-linked section windows. Without this function, the static HTML shell would never become the actual portfolio.